2110573 PATTERN RECOGNITION

Homework 1: Clustering and Regression

Worralop Srichainont

January 14, 2026

Metrics

In a population where the amount of cats is equal to the amount of dogs. Considering the following classification results from a classifier.

Model A	Predicted dog	Predicted cat
Actual dog	30	20
Actual cat	10	40

T1. What is the accuracy of Model A?

Sol. Accuracy of the model A is the total correct predictions divided by the total prediction. In this case, we assume that dogs as positive class.

$$\text{Accuracy} = \frac{TP + TN}{Total}$$

$$\text{Accuracy} = \frac{30 + 40}{100}$$

$$\text{Accuracy} = 70.00\%$$

Ans. Model A has 70.00% accuracy.

T2. Consider cats as class 1 (positive) and dogs as class 0 (negative), calculate the precision, recall, and F1.

Sol. Consider true positive (TP), false positive (FP), true negative (TN), and false negative (FN) of the model A.

TP means predicted cat, actual cat. (40 predictions)

FP means predicted cat, actual dog. (20 predictions)

TN means predicted dog, actual dog. (30 predictions)

FN means predicted dog, actual cat. (10 predictions)

Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Precision} = \frac{40}{40 + 20} \approx 66.67\%$$

Recall

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{Recall} = \frac{40}{40 + 10} = 80.00\%$$

F1 Score

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{F1} = 2 \times \frac{\frac{2}{3} \times 0.80}{\frac{2}{3} + 0.80} \approx 0.73$$

Ans. Model A has 66.67% precision, 80.00% recall, and 0.73 F1 score.

T3. Consider class cat as class 0 (negative) and class dog as class 1 (positive), calculate the precision, recall, and F1.

It is important to specify the positive class when you calculate precision, recall, and F1. If there are more than two classes, it is usually done in a one-versus-all setting where one class is considered positive and the rest of the classes are considered negative.

Sol. Consider true positive (TP), false positive (FP), true negative (TN), and false negative (FN) of the model A.

TP means predicted dog, actual dog. (30 predictions)

FP means predicted dog, actual cat. (10 predictions)

TN means predicted cat, actual cat. (40 predictions)

FN means predicted cat, actual dog. (20 predictions)

Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Precision} = \frac{30}{30 + 10} = 75.00\%$$

Recall

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{Recall} = \frac{30}{30 + 20} = 60.00\%$$

F1 Score

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{F1} = 2 \times \frac{0.75 \times 0.60}{0.75 + 0.60} \approx 0.67$$

Ans. Model A has 75.00% precision, 60.00% recall, and 0.67 F1 score.

T4. Now consider a lopsided population where there are 80% cats. What is the accuracy of Model A? Using dog as the positive class, what is the precision, recall, and F1? Explain how and why these numbers change (or does not change) from the previous questions.

Sol. First of all, we need to adjust the ratio of population from 50% : 50% to 20% : 80%.

Model A	Predicted dog	Predicted cat
Actual dog	$30 \times \frac{20\%}{50\%} = 12$	$20 \times \frac{20\%}{50\%} = 8$
Actual cat	$10 \times \frac{80\%}{50\%} = 16$	$40 \times \frac{80\%}{50\%} = 64$

Therefore, true positive (TP), false positive (FP), true negative (TN), and false negative (FN) of the model A are:

TP means predicted dog, actual dog. (12 predictions)

FP means predicted dog, actual cat. (16 predictions)

TN means predicted cat, actual cat. (64 predictions)

FN means predicted cat, actual dog. (8 predictions)

Next, calculate accuracy, precision, recall, and F1 score of the new population.

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{Total}$$

$$\text{Accuracy} = \frac{12 + 64}{100} = 76.00\%$$

Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Precision} = \frac{12}{12 + 16} \approx 42.86\%$$

Recall

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{Recall} = \frac{12}{12 + 8} = 60.00\%$$

F1 Score

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{F1} = 2 \times \frac{\frac{3}{7} \times 0.60}{\frac{3}{7} + 0.60} = 0.50$$

Ans. The accuracy of the model A increases from 70.00% to 76.00% due to increasing cat population and model's ability to predict cat prediction accurately.

The precision of the model A decreases from 75.00% to 42.86% due to increasing cat population which increases false positive prediction.

The recall of the model A remains the same (60.00%), because the ability to predict the dog correctly remains the same.

The F1 score of the model A decreases from 0.67 to 0.50 due to decreasing precision of the model.

Hello Clustering

Recall from lecture that K-means has two main steps: the points assignment step, and the mean update step. After the initialization of the centroids, we assign each data point to a centroid. Then, each centroids are updated by re-estimating the means.

Concretely, if we are given N data points, x_1, x_2, ..., x_N, and we would like to form K clusters. We do the following;

1. **Initialization:** Pick K random data points as K centroid locations c_1, c_2, ..., c_K.
2. **Assign:** For each data point k, find the closest centroid. Assign that data point to the centroid. The distance used is typically Euclidean distance.
3. **Update:** For each centroid, calculate the mean from the data points assigned to it.
4. **Repeat:** repeat step 2 and 3 until the centroids stop changing (convergence).

Given the following data points in x-y coordinates (2 dimensional)

x	y
1	2
3	3
2	2
8	8
6	6
7	7
-3	-3
-2	-4
-7	-7

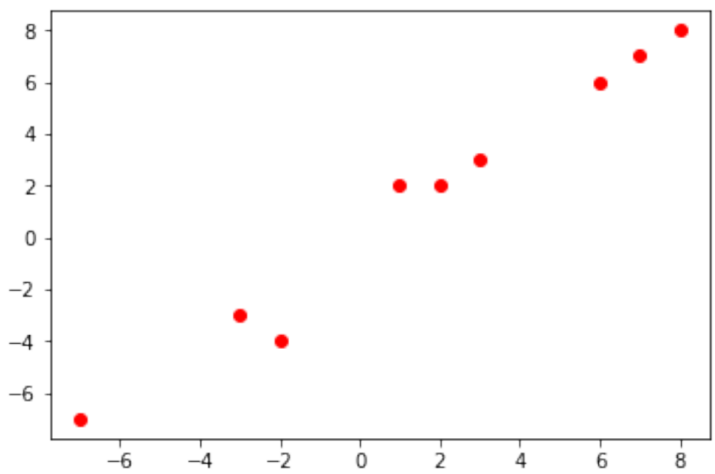

Figure 1: Data points on (x, y) coordinates

T5. If the starting points are (3,3), (2,2), and (-3,-3). Describe each assign and update step. What are the points assigned? What are the updated centroids? You may do this calculation by hand or write a program to do it.

<u>Sol.</u> <u>First Iteration</u>

First, we calculate distance between each point to each selected centroids c_1, c_2, and c_3.

Table 1: The first iteration of K-means Clustering Algorithm

x	y	$c_1 = (3,3)$	$c_2 = (2,2)$	$c_3 = (-3,-3)$	Cluster
1	2	2.24	1.00	6.40	Cluster 2
3	3	0.00	1.41	8.49	Cluster 1
2	2	1.41	0.00	7.07	Cluster 2
8	8	7.07	8.49	15.56	Cluster 1
6	6	4.24	5.66	12.73	Cluster 1
7	7	5.66	7.07	14.14	Cluster 1
-3	-3	8.49	7.07	0.00	Cluster 3
-2	-4	8.60	7.21	1.41	Cluster 3
-7	-7	14.14	12.73	5.66	Cluster 3

Then assign each point to the closest centroid.

- **Cluster 1** contains $\{(3,3),(8,8),(6,6),(7,7)\}$
- **Cluster 2** contains $\{(1,2),(2,2)\}$
- **Cluster 3** contains $\{(-3,-3),(-2,-4),(-7,-7)\}$

Then re-calculate the centroid of each clusters by using the formula.

$$\text{Centroid} = \left(\frac{x_1 + x_2 + ... + x_N}{N}, \frac{y_1 + y_2 + ... + y_N}{N}\right)$$

These are re-calculated centroids of each cluster.

$$\text{Centroid 1 } (c_1) = \left(\frac{3+8+6+7}{4}, \frac{3+8+6+7}{4}\right)$$
$$\text{Centroid 1 } (c_1) = (6.00, 6.00)$$

$$\text{Centroid 2 } (c_2) = \left(\frac{1+2}{2}, \frac{2+2}{2}\right)$$
$$\text{Centroid 2 } (c_2) = (1.50, 2.00)$$

$$\text{Centroid 3 } (c_3) = \left(\frac{(-3)+(-2)+(-7)}{3}, \frac{(-3)+(-4)+(-7)}{3}\right)$$
$$\text{Centroid 3 } (c_3) = (-4.00, -4.67)$$

Second Iteration

First, we calculate distance between each point to each new centroids c_1, c_2, and c_3.

Table 2: The second iteration of K-means Clustering Algorithm

x	y	$c_1 = (6.00, 6.00)$	$c_2 = (1.50, 2.00)$	$c_3 = (-4.00, -4.67)$	Cluster
1	2	6.40	0.50	8.33	Cluster 2
3	3	4.24	1.80	10.38	Cluster 2
2	2	5.66	0.50	8.97	Cluster 2
8	8	2.83	8.85	17.45	Cluster 1
6	6	0.00	6.02	14.62	Cluster 1
7	7	1.41	7.43	16.03	Cluster 1
-3	-3	12.73	6.73	1.94	Cluster 3
-2	-4	12.81	6.95	2.11	Cluster 3
-7	-7	18.38	12.38	3.80	Cluster 3

Then assign each point to the closest centroid.

- **Cluster 1** contains $\{(8, 8), (6, 6), (7, 7)\}$
- **Cluster 2** contains $\{(1, 2), (3, 3), (2, 2)\}$
- **Cluster 3** contains $\{(-3, -3), (-2, -4), (-7, -7)\}$

Then re-calculate the centroid of each clusters by using the formula.

$$\text{Centroid} = \left(\frac{x_1 + x_2 + ... + x_N}{N}, \frac{y_1 + y_2 + ... + y_N}{N} \right)$$

These are re-calculated centroids of each cluster.

$$\text{Centroid 1 } (c_1) = \left(\frac{8 + 6 + 7}{3}, \frac{8 + 6 + 7}{3} \right)$$
$$\text{Centroid 1 } (c_1) = (7.00, 7.00)$$

$$\text{Centroid 2 } (c_2) = \left(\frac{1 + 3 + 2}{3}, \frac{2 + 3 + 2}{3} \right)$$
$$\text{Centroid 2 } (c_2) = (2.00, 2.33)$$

$$\text{Centroid 3 } (c_3) = \left(\frac{(-3) + (-2) + (-7)}{3}, \frac{(-3) + (-4) + (-7)}{3} \right)$$
$$\text{Centroid 3 } (c_3) = (-4.00, -4.67)$$

Third Iteration

First, we calculate distance between each point to each new centroids c_1, c_2, and c_3.

Table 3: The third iteration of K-means Clustering Algorithm

x	y	$c_1 = (7.00, 7.00)$	$c_2 = (2.00, 2.33)$	$c_3 = (-4.00, -4.67)$	Cluster
1	2	7.81	1.05	8.33	Cluster 2
3	3	5.66	1.20	10.38	Cluster 2
2	2	7.07	0.33	8.97	Cluster 2
8	8	1.41	8.25	17.45	Cluster 1
6	6	1.41	5.43	14.62	Cluster 1
7	7	0.00	6.84	16.03	Cluster 1
-3	-3	14.14	7.31	1.94	Cluster 3
-2	-4	14.21	7.49	2.11	Cluster 3
-7	-7	19.80	12.97	3.80	Cluster 3

Then assign each point to the closest centroid.

- **Cluster 1** contains $\{(8, 8), (6, 6), (7, 7)\}$
- **Cluster 2** contains $\{(1, 2), (3, 3), (2, 2)\}$
- **Cluster 3** contains $\{(-3, -3), (-2, -4), (-7, -7)\}$

Then re-calculate the centroid of each clusters by using the formula.

$$\text{Centroid} = \left(\frac{x_1 + x_2 + ... + x_N}{N}, \frac{y_1 + y_2 + ... + y_N}{N} \right)$$

These are re-calculated centroid of each cluster.

$$\text{Centroid 1 } (c_1) = \left(\frac{8 + 6 + 7}{3}, \frac{8 + 6 + 7}{3} \right)$$
$$\text{Centroid 1 } (c_1) = (7.00, 7.00)$$

$$\text{Centroid 2 } (c_2) = \left(\frac{1 + 3 + 2}{3}, \frac{2 + 3 + 2}{3} \right)$$
$$\text{Centroid 2 } (c_2) = (2.00, 2.33)$$

$$\text{Centroid 3 } (c_3) = \left(\frac{(-3) + (-2) + (-7)}{3}, \frac{(-3) + (-4) + (-7)}{3} \right)$$
$$\text{Centroid 3 } (c_3) = (-4.00, -4.67)$$

Since centroid of each each cluster doesn't change, the algorithm is terminated.

<u>Ans.</u> Each cluster has centroid and members as following.

- **Cluster 1** has centroid at $(7.00, 7.00)$ and contains $\{(8, 8), (6, 6), (7, 7)\}$
- **Cluster 2** has centroid at $(2.00, 2.33)$ and contains $\{(1, 2), (3, 3), (2, 2)\}$
- **Cluster 3** has centroid at $(-4.00, -4.67)$ and contains $\{(-3, -3), (-2, -4), (-7, -7)\}$

T6. If the starting points are (-3,-3), (2,2), and (-7,-7), what happens?

Sol. First Iteration

First, we calculate distance between each point to each selected centroids c_1, c_2, and c_3.

Table 4: The first iteration of K-means Clustering Algorithm

x	y	$c_1 = (-3, -3)$	$c_2 = (2, 2)$	$c_3 = (-7, -7)$	Cluster
1	2	6.40	1.00	12.04	Cluster 2
3	3	8.49	1.41	14.14	Cluster 2
2	2	7.07	0.00	12.73	Cluster 2
8	8	15.56	8.49	21.21	Cluster 2
6	6	12.73	5.66	18.38	Cluster 2
7	7	14.14	7.07	19.80	Cluster 2
-3	-3	0.00	7.07	5.66	Cluster 1
-2	-4	1.41	7.21	5.83	Cluster 1
-7	-7	5.66	12.73	0.00	Cluster 3

Then assign each point to the closest centroid.

- **Cluster 1** contains $\{(-3, -3), (-2, -4)\}$
- **Cluster 2** contains $\{(1, 2), (3, 3), (2, 2), (8, 8), (6, 6), (7, 7)\}$
- **Cluster 3** contains $\{(-7, -7)\}$

Then re-calculate the centroid of each clusters by using the formula.

$$\text{Centroid} = \left(\frac{x_1 + x_2 + ... + x_N}{N}, \frac{y_1 + y_2 + ... + y_N}{N} \right)$$

These are re-calculated centroids of each cluster.

$$\text{Centroid 1 } (c_1) = \left(\frac{(-3) + (-2)}{2}, \frac{(-3) + (-4)}{2} \right)$$
$$\text{Centroid 1 } (c_1) = (-2.50, -3.50)$$

$$\text{Centroid 2 } (c_2) = \left(\frac{1 + 3 + 2 + 8 + 6 + 7}{6}, \frac{2 + 3 + 2 + 8 + 6 + 7}{6} \right)$$
$$\text{Centroid 2 } (c_2) = (4.50, 4.67)$$

$$\text{Centroid 3 } (c_3) = \left(\frac{-7}{1}, \frac{-7}{1} \right)$$
$$\text{Centroid 3 } (c_3) = (-7.00, -7.00)$$

Second Iteration

First, we calculate distance between each point to each new centroids c_1, c_2, and c_3.

Table 5: The second iteration of K-means Clustering Algorithm

x	y	$c_1 = (-2.50, -3.50)$	$c_2 = (4.50, 4.67)$	$c_3 = (-7.00, -7.00)$	Cluster
1	2	6.52	4.40	12.04	Cluster 2
3	3	8.51	2.24	14.14	Cluster 2
2	2	7.11	3.66	12.73	Cluster 2
8	8	15.57	4.83	21.21	Cluster 2
6	6	12.75	2.01	18.38	Cluster 2
7	7	14.16	3.42	19.80	Cluster 2
-3	-3	0.71	10.73	5.66	Cluster 1
-2	-4	0.71	10.83	5.83	Cluster 1
-7	-7	5.70	16.38	0.00	Cluster 3

Then assign each point to the closest centroid.

- **Cluster 1** contains $\{(-3, -3), (-2, -4)\}$
- **Cluster 2** contains $\{(1, 2), (3, 3), (2, 2), (8, 8), (6, 6), (7, 7)\}$
- **Cluster 3** contains $\{(-7, -7)\}$

Then re-calculate the centroid of each clusters by using the formula.

$$\text{Centroid} = \left(\frac{x_1 + x_2 + ... + x_N}{N}, \frac{y_1 + y_2 + ... + y_N}{N} \right)$$

These are re-calculated centroids of each cluster.

$$\text{Centroid 1 } (c_1) = \left(\frac{(-3) + (-2)}{2}, \frac{(-3) + (-4)}{2} \right)$$
$$\text{Centroid 1 } (c_1) = (-2.50, -3.50)$$

$$\text{Centroid 2 } (c_2) = \left(\frac{1 + 3 + 2 + 8 + 6 + 7}{6}, \frac{2 + 3 + 2 + 8 + 6 + 7}{6} \right)$$
$$\text{Centroid 2 } (c_2) = (4.50, 4.67)$$

$$\text{Centroid 3 } (c_3) = \left(\frac{-7}{1}, \frac{-7}{1} \right)$$
$$\text{Centroid 3 } (c_3) = (-7.00, -7.00)$$

<u>Ans.</u> Each cluster has centroid and members as following.

- **Cluster 1** has centroid at $(-2.50, -3.50)$ and contains $\{(-3, -3), (-2, -4)\}$
- **Cluster 2** has centroid at $(4.50, 4.67)$ and contains $\{(1, 2), (3, 3), (2, 2), (8, 8), (6, 6), (7, 7)\}$
- **Cluster 3** has centroid at $(-7.00, -7.00)$ and contains $\{(-7, -7)\}$

As we choose different initial centroid points, sometimes it gives different clustering results.

T7. Between the two starting set of points in the previous two questions, which one do you think is better? How would you measure the 'goodness' quality of a set of starting points?

In general, it is important to try different sets of starting points when doing k-means.

Ans. This figure below shows the data points clustering on problem **T5**.

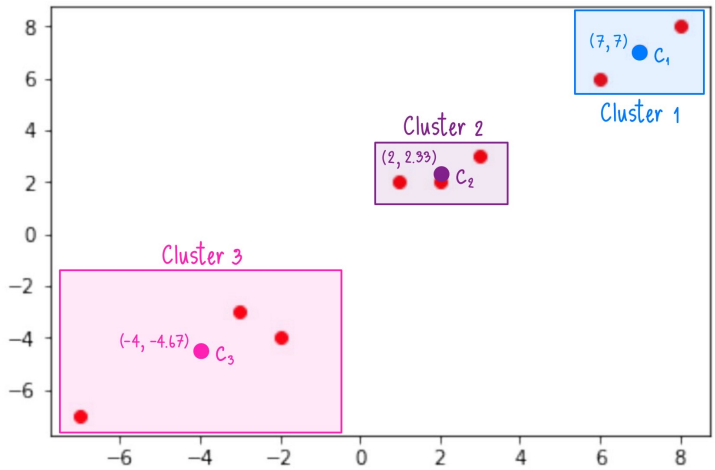

Figure 2: Data points clustering on problem **T5**

And this figure below shows the data points clustering on problem **T6**.

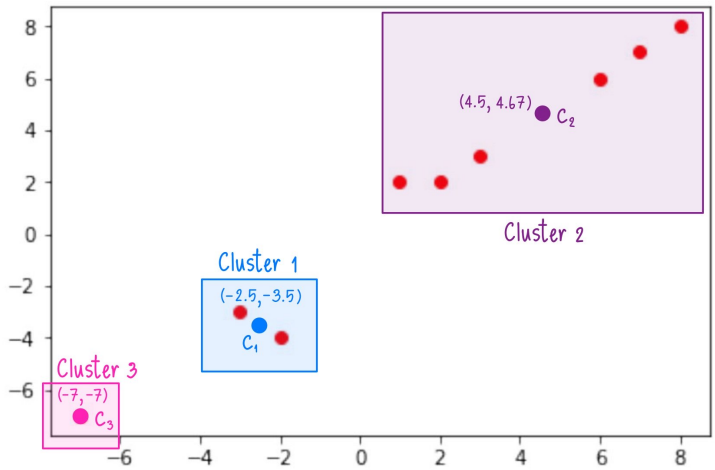

Figure 3: Data points clustering on problem **T6**

To determine which has better clustering, consider these 2 factors as following.

- The data point should be as near as possible to their cluster.
- The data point should be as far as possible to the other cluster.

In my opinion, data points clustering on problem **T5** is better, because the clusters on problem **T5** are not too large compared to the cluster 2 of problem **T6**.

My heart will go on

In this part of the exercise we will work on the Titanic dataset provided by Kaggle. The Titanic dataset contains information of the passengers boarding the Titanic on its final voyage. We will work on predicting whether a given passenger will survive the trip.

```
TRAIN_URL = "http://s3.amazonaws.com/assets.datacamp.com/course/Kaggle/
    train.csv"
TEST_URL = "http://s3.amazonaws.com/assets.datacamp.com/course/Kaggle/
    test.csv"

train = pd.read_csv(TRAIN_URL)
test = pd.read_csv(TEST_URL)
```

Code 1: Import data using Pandas

Both train and test are dataframes. Use the function `train.head()` and `train.tail()` to explore the data. What do you see?

Use the function `describe()` to get a better understanding of the data. You can read the meaning of the data fields at `https://www.kaggle.com/c/titanic/data`

Looking at the data, you will notice a lot of missing values. For example, some age is NaN. This is normal for real world data to have some missing values.

There are several ways to handle missing values. The simplest is to throw away any rows that have missing values. However, this usually reduce the amount of training data you have. Another method is to guess what the missing value should be. The simplest guess is to use the Median or Mode of the data. For this exercise we will proceed with this.

T8. Perform the following preprocessing steps on the dataset.

T8.1. What is the median age of the training set?

Sol. To calculate median of age column, simply use the following command.

```
median_age = train["Age"].median()
print(f"The median age is {median_age}.")
```

Code 2: Calculate median value of the age column

Ans. The median of the age column is 28 years.

T8.2. Fill the missing values with the median.

Sol. To fill the median value of age to missing values in training dataset, simply use the following command.

```
train["Age"] = train["Age"].fillna(median_age)
```

Code 3: Fill missing age value with median value

T9. Perform the following preprocessing steps on the dataset.

T9.1. Some fields like `Embarked` are categorical. They need to be converted to numbers first. We will represent `S` with `0`, `C` with `1`, and `Q` with `2`.

Sol. Convert categorical value on Embarked column from `S`, `C`, and `Q` to `0`, `1`, and `2` respectively.

```
train.loc[train["Embarked"] == "S", "Embarked"] = 0
train.loc[train["Embarked"] == "C", "Embarked"] = 1
train.loc[train["Embarked"] == "Q", "Embarked"] = 2
```

Code 4: Convert categorical value on `Embarked` column

T9.2. What is the mode of `Embarked`?

Sol. Calculate mode value of the `Embarked` column. Note that `mode()` returns `Series` object, not a single value.

```
mode_embarked = train["Embarked"].mode()[0]
print(f"The mode of embarked is {mode_embarked}.")
```

Code 5: Calculate mode value of the `Embarked` column

Ans. The mode value of the `Embarked` column is 0 (Southampton).

T9.3. Fill the missing values with the mode.

Sol. Fill the missing value on Embarked column with the mode.

```
train["Embarked"] = train["Embarked"].fillna(mode_embarked)
```

Code 6: Fill the missing embarked value with mode value

T9.4. Do the same for `Sex`.

Sol. Convert categorical value on `Embarked` column from `male`, `female` to `0` and `1` respectively.

```
# Convert categorical value to 0 and 1
train.loc[train["Sex"] == "male", "Sex"] = 0
train.loc[train["Sex"] == "female", "Sex"] = 1

# Calculate mode value of the Sex column.
mode_sex = train["Sex"].mode()[0]
print(f"The mode of sex is {mode_sex}.")

# Fill missing value with mode value
train["Sex"] = train["Sex"].fillna(mode_sex)
```

Code 7: Repeat the process on the `Sex` column.

Ans. The mode value of the `Sex` column is 0 (male).

T10. Write a linear regression classifier using gradient descent as learned in class. Use `Pclass`, `Sex`, `Age`, and `Embarked` as input features with threshold value 0.5.

Sol. For this problem, set the configuration as following.

```
INPUT_FEATURES = ["Pclass", "Sex", "Age", "Embarked"]
OUTPUT_FEATURE = "Survived"
OUTPUT_FILENAME = "submission.csv"
ID_COLUMN = "PassengerId"

CLASSIFICATION_THRESHOLD = 0.50
```

Code 8: Configuration for problem **T10**

Train a linear regression model from `scikit-learn` library.

```
# Extract data from train dataset
X = train[INPUT_FEATURES]
y = train[OUTPUT_FEATURE]

# Train the linear regression model
model = LinearRegression()
model.fit(X, y)
```

Code 9: Train a linear regression model

Use pre-trained model from the previous step to predict test dataset.

```
# Extract data from test dataset
X_test = test[INPUT_FEATURES]

# Input test dataset into pre-trained linear regression model.
raw_predictions = model.predict(X_test)
```

Code 10: Prediction on test dataset using linear regression model

Create a `Pandas DataFrame` containing `PassengerId` and `Survived` columns for submission file in Kaggle competition.

```
# Initialize empty DataFrame
submission = pd.DataFrame()

# Add passenger ID column to the DataFrame
submission[ID_COLUMN] = test[ID_COLUMN]

# Add raw prediction value to the DataFrame
submission[OUTPUT_FEATURE] = raw_predictions
```

Code 11: Create `Pandas DataFrame` for Kaggle competition

Since it is classification task, the predicted value has to be `0` or `1` only. Therefore we are applying the threshold as following:

- If the predicted value is less than `0.5`, change them to `0`.
- If the predicted value is more than or equal to `0.5`, change them to `1`.

```
submission[OUTPUT_FEATURE] = (
    submission[OUTPUT_FEATURE] >= CLASSIFICATION_THRESHOLD
).astype(int)
```

Code 12: Applying threshold for classification

Finally, save Kaggle submission file as `csv` file.

```
submission.to_csv(OUTPUT_FILENAME, index=False)
```

Code 13: Save submission `csv` file.

T11. To submit your prediction, you must first sign-up for an account on `https://www.kaggle.com`. Click participate to the competition at `https://www.kaggle.com/c/titanic/` then submit your csv file for the score. Submit a screenshot of your submission (with the scores).

Ans. The submission score from problem **T10** is 0.76315.

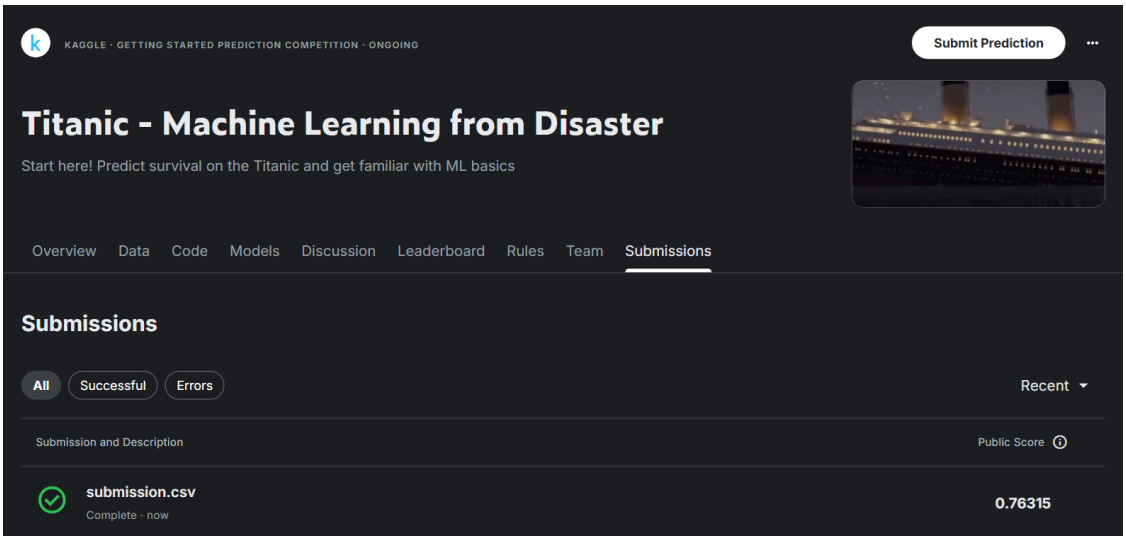

Figure 4: Submission score of the problem **T10**

T12. Try adding some higher order features to your training (${x_1}^2$, x_1x_2, ...). Does this model has better accuracy on the training set? How does it perform on the test set?

Sol. For this problem, set the configuration as following.

```
INPUT_FEATURES = ["Pclass", "Sex", "Age", "Embarked"]
OUTPUT_FEATURE = "Survived"
OUTPUT_FILENAME = "submission_high_order.csv"
ID_COLUMN = "PassengerId"

CLASSIFICATION_THRESHOLD = 0.50
```

Code 14: Configuration for problem **T12**

Use `PolynomialFeatures()` function from `scikit-learn` library to automatically apply higher order features on dataset.

```
# Extract data from train dataset
X = train[INPUT_FEATURES]
y = train[OUTPUT_FEATURE]

# Add high order features to the train dataset
X_high_order = PolynomialFeatures(degree=2).fit_transform(X)

# Extract data from test dataset
X_test = test[INPUT_FEATURES]

# Add high order features to the test dataset.
X_test_high_order = PolynomialFeatures(degree=2).fit_transform(
    X_test)
```

Code 15: Applying higher order features on both datasets

Train a linear regression model from `scikit-learn` library with higher order features.

```
# Train the linear regression model with high order features.
model = LinearRegression()
model.fit(X_high_order, y)
```

Code 16: Train a linear regression model with higher order features

Use pre-trained model from the previous step to predict test dataset. Make sure to apply higher order features on test dataset too.

```
# Input test dataset with high order features into pre-trained
    linear regression model.
raw_predictions = model.predict(X_test_high_order)
```

Code 17: Prediction on test dataset using linear regression model

Create a `Pandas DataFrame` containing `PassengerId` and `Survived` columns for submission file in Kaggle competition.

```
# Initialize empty DataFrame
submission_high_order = pd.DataFrame()

# Add passenger ID column to the DataFrame
submission_high_order[ID_COLUMN] = test[ID_COLUMN]

# Add raw prediction value to the DataFrame
submission_high_order[OUTPUT_FEATURE] = raw_predictions
```

Code 18: Create `Pandas DataFrame` for Kaggle competition

Since it is classification task, the predicted value has to be `0` or `1` only. Therefore we are applying the threshold as following:

- If the predicted value is less than `0.5`, change them to `0`.
- If the predicted value is more than or equal to `0.5`, change them to `1`.

```
submission_high_order[OUTPUT_FEATURE] = (
    submission_high_order[OUTPUT_FEATURE] >=
   CLASSIFICATION_THRESHOLD
).astype(int)
```

Code 19: Applying threshold for classification

Finally, save Kaggle submission file as `csv` file.

```
submission_high_order.to_csv(OUTPUT_FILENAME, index=False)
```

Code 20: Save submission `csv` file.

Ans. The submission score from problem **T12** is 0.77980, which is higher than score from problem **T10** (0.76315).

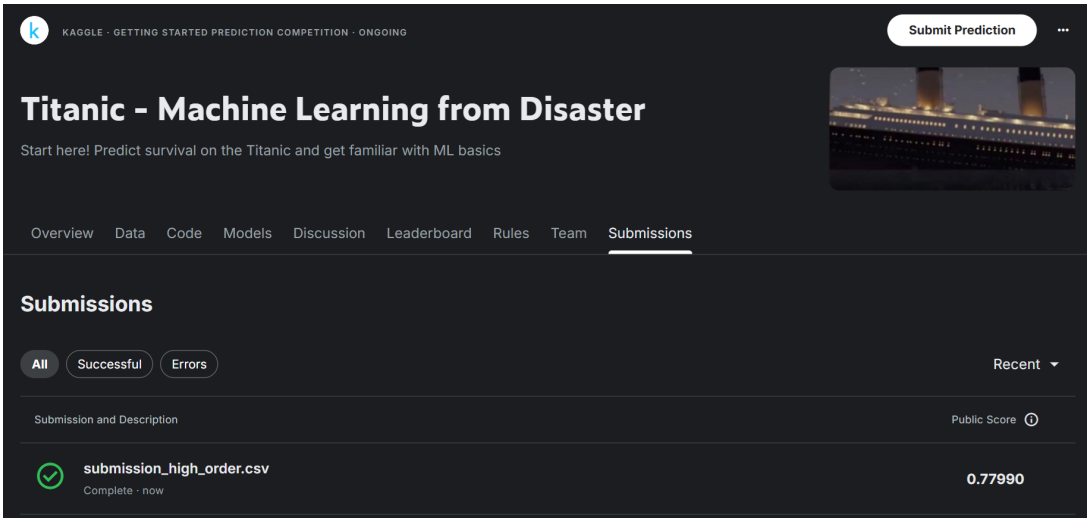

Figure 5: Submission score of the problem **T12**

T13. What happens if you reduce the amount of features to just `Sex` and `Age`?

Sol. For this problem, set the configuration as following.

```
INPUT_FEATURES = ["Sex", "Age"]
OUTPUT_FEATURE = "Survived"
OUTPUT_FILENAME = "submission_reduced.csv"
ID_COLUMN = "PassengerId"

CLASSIFICATION_THRESHOLD = 0.50
```

Code 21: Configuration for problem **T13**

Train a linear regression model from `scikit-learn` library.

```
# Extract data from train dataset
X = train[INPUT_FEATURES]
y = train[OUTPUT_FEATURE]

# Train the linear regression model
model = LinearRegression()
model.fit(X, y)
```

Code 22: Train a linear regression model

Use pre-trained model from the previous step to predict test dataset.

```
# Extract data from test dataset
X_test = test[INPUT_FEATURES]

# Input test dataset into pre-trained linear regression model.
raw_predictions = model.predict(X_test)
```

Code 23: Prediction on test dataset using linear regression model

Create a `Pandas DataFrame` containing `PassengerId` and `Survived` columns for submission file in Kaggle competition.

```
# Initialize empty DataFrame
submission_reduced = pd.DataFrame()

# Add passenger ID column to the DataFrame
submission_reduced[ID_COLUMN] = test[ID_COLUMN]

# Add raw prediction value to the DataFrame
submission_reduced[OUTPUT_FEATURE] = raw_predictions
```

Code 24: Create `Pandas DataFrame` for Kaggle competition

Since it is classification task, the predicted value has to be `0` or `1` only. Therefore we are applying the threshold as following:

- If the predicted value is less than `0.5`, change them to `0`.
- If the predicted value is more than or equal to `0.5`, change them to `1`.

```
submission_reduced[OUTPUT_FEATURE] = (
    submission_reduced[OUTPUT_FEATURE] >= CLASSIFICATION_THRESHOLD
).astype(int)
```

Code 25: Applying threshold for classification

Finally, save Kaggle submission file as `csv` file.

```
submission_reduced.to_csv(OUTPUT_FILENAME, index=False)
```

Code 26: Save submission `csv` file.

<u>Ans.</u> The submission score from problem **T13** is 0.76555, which is slightly higher than score from problem **T10** (0.76315).

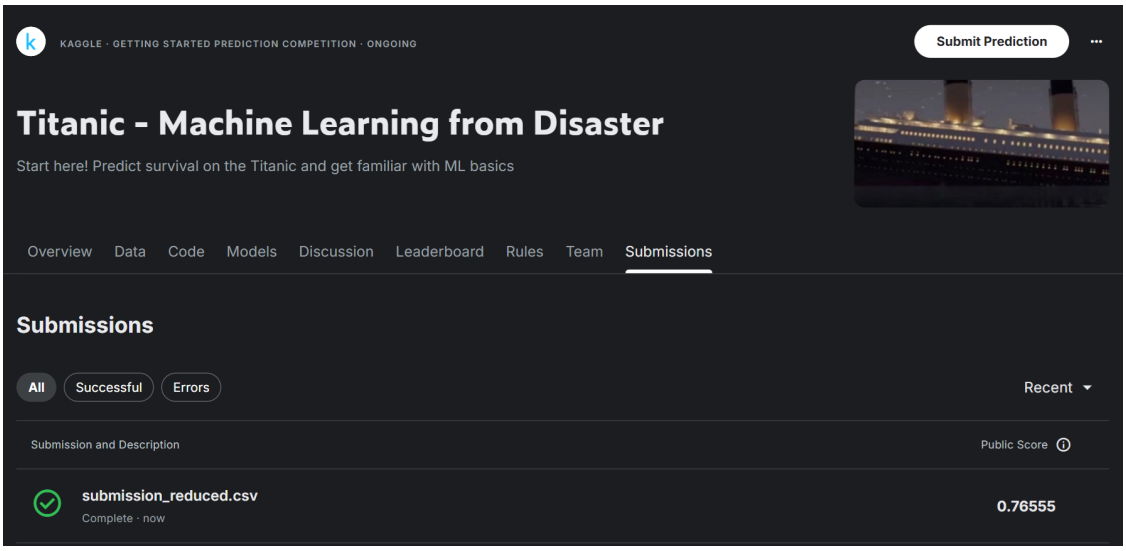

Figure 6: Submission score of the problem **T13**

Additional Resources

Jupyter Notebook for problem **T8** to **T13**: `https://colab.research.google.com/drive/1R_SxhXRS_jfXk-5L-U85Kp-usC1y01sx`